

Slides

Slides — Docker: Containers e Entrega

Slide 1 — Capa

Slide (título e informações): - Docker: Containers e Entrega - Docker & Docker Compose - Grupo: [nome do grupo] - Integrantes: [nomes]

Notas do apresentador: “Olá, nosso grupo vai apresentar Docker e Docker Compose — ferramentas essenciais para containers e entrega de aplicações.”

Slide 2 — O Problema

Slide (bullet points): - “Funciona na minha máquina” — ambiente inconsistente entre devs - Setup manual demorado: instalar banco, cache, fila, configurar versões - Conflitos entre projetos: Node 18 vs Node 20, Python 3.9 vs 3.12

Notas do apresentador: “Quanto aqui já ouviram ‘funciona na minha máquina’? Ou já perderam horas configurando ambiente em um projeto novo? Imagine uma equipe de 5 pessoas: Maria usa Windows, João usa Mac, você usa Linux. O banco é PostgreSQL, o cache é Redis, o backend é Node.js, o frontend é React. Sem Docker, cada pessoa passa horas configurando. Com Docker Compose, um git pull + docker compose up -d e tudo sobe em segundos.”

Slide 3 — Docker: Conceitos

Slide (bullet points): - Container: processo isolado (PID, NET, MNT) com recursos limitados (CPU, RAM) - Imagem: template imutável em camadas (layer cache) - Dockerfile: receita para construir a imagem (FROM, RUN, COPY, CMD) - Docker Hub: registry público de imagens

Diagrama sugerido:

VM: [App] [App] cada VM tem SO próprio (GB)
[SO] [SO]
[Hypervisor]

Container: [App] [App] compartilha kernel do host (MB)
[Docker]
[SO do host]

Notas do apresentador: “Diferente de máquinas virtuais que virtualizam hardware inteiro — cada VM com seu próprio SO convidado ocupando gigabytes — containers compartilham o kernel do host. Um container é apenas um processo isolado que enxerga seu próprio filesystem e rede, mas iniciado em segundos. Cada instrução de um Dockerfile gera uma camada cacheável: se você muda o código mas não as dependências, o Docker reusa as camadas anteriores e o build é quase instantâneo.”

Slide 4 — Docker Compose

Slide (bullet points): - Projetos reais precisam de múltiplos serviços: app + db + cache + fila - Compose coordena tudo com um único arquivo YAML - docker compose up -d → sobe a stack inteira - docker compose down -v → derruba tudo, incluindo volumes

Exemplo de YAML:

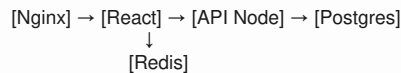
```
services:
  app:
    build: .
    ports: ["3000:3000"]
  db:
    image: postgres:16
    volumes: ["pgdata:/var/lib/postgresql/data"]
```

Notas do apresentador: “Gerenciar múltiplos containers manualmente é inviável — você teria que criar redes, volumes, iniciar cada container na ordem certa, expor portas. O Compose declara tudo em um arquivo YAML e um comando faz o trabalho pesado. Outro comando derruba tudo de forma limpa, inclusive os volumes de dados com a flag -v.”

Slide 5 — Sistemas para Internet

Slide (bullet points): - **Paridade dev/prod:** mesmo container no notebook e no servidor - **Onboarding:** tutorial vira “instale Docker e rode `docker compose up`” - **Microserviços:** cada serviço escala independente - **CI/CD:** pipeline usa containers idênticos para testar e promover - **Isolamento de versões:** Node 18 e Node 20 no mesmo PC sem conflito

Diagrama sugerido:



Notas do apresentador: “Paridade dev/prod: o mesmo container que você sobe no seu computador vai exatamente igual para o servidor de produção. Nada de configurações diferentes, versões diferentes de bibliotecas. Onboarding: quando chega um dev novo, o tutorial de setup costumava ser um documento de 20 páginas — instale PostgreSQL, Node, Redis. Com Docker, o tutorial vira: ‘instale Docker e rode `docker compose up`’. Minutos, não dias. Microserviços: cada serviço — autenticação, pagamento, catálogo — roda em seu próprio container e escala sozinho. E CI/CD: pipelines sobem containers idênticos aos de produção para testar e promovem a mesma imagem.”

Slide 6 — Instalação

Slide (bullet points): - Script oficial (qualquer Linux): `curl -fsSL https://get.docker.com | sh` - Ubuntu/Debian: `sudo apt install docker.io` - Pós-instalação (obrigatório): `bash sudo usermod -aG docker $USER newgrp docker` - Verificar: `docker --version` e `docker run hello-world` - Compose plugin já vem incluso desde 2023 - Verificar: `docker compose version`

Notas do apresentador: “No Linux, o jeito mais simples é o script oficial — ele detecta a distribuição e instala tudo. Após instalar, um passo obrigatório: adicionar seu usuário ao grupo docker. Se pular isso, você vai ter que usar `sudo` em todo comando docker. O `newgrp docker` ativa o grupo na sessão atual sem precisar relogar. Para verificar se deu certo, `docker run hello-world` baixa uma imagem de teste e roda um container. Desde 2023, o Docker Compose vem como plugin integrado ao Docker CLI — não precisa instalar separado.”

Slide 7 — Comandos Essenciais

Slide (tabelas):

Docker: | Comando | O que faz | |—| | `docker pull nginx` | Baixa imagem | | `docker build -t app .` | Constrói imagem | | `docker run -d -p 3000:3000 app` | Cria e inicia container | | `docker ps -a` | Lista containers | | `docker stop app` && `docker rm app` | Para e remove | | `docker exec -it app sh` | Executa comando | | `docker logs -f app` | Logs em tempo real | | `docker system prune -a` | Limpeza geral |

Docker Compose: | Comando | O que faz | |—| | `docker compose up -d` | Sobe a stack | | `docker compose up -d --build` | Builda e sobe | | `docker compose down -v` | Derruba + volumes | | `docker compose ps` | Status dos serviços | | `docker compose logs -f` | Logs de todos | | `docker compose exec app sh` | Shell no serviço |

Flags importantes: -d (background), -p (porta), -v (volume), -e (ambiente), -it (interativo), --name (nome), --rm (auto-remover).

Notas do apresentador: “Esses são os comandos que você vai usar 90% do tempo. No Docker, o fluxo básico é: baixar ou construir uma imagem, criar um container com `docker run`, e gerenciar com `ps`, `stop`, `rm`. No Compose, o fluxo é ainda mais simples: `docker compose up -d` para subir tudo, `docker compose down` para derrubar. As flags do `docker run` são importantes: -d roda em background, -p mapeia a porta do container para o host, -v monta um volume para dados persistirem, -e define variáveis de ambiente.”

Slide 8 — Caso Real: Arquitetura

Slide (bullet points): - Repositório: github.com/VDCSS/demo-Docker - **O que faz:** API Node.js que consulta a hora no PostgreSQL e retorna JSON - Rotas: GET / (hora do banco) e GET /health (saúde do serviço) - Stack: app (Node.js 20 Alpine) + db (PostgreSQL 16 Alpine) - Rede interna isolada, comunicação via nome do serviço (db) - Volume nomeado para dados persistentes

Diagrama:

[app:3000] ↔ [db:5432]
rede interna (DNS = nome do serviço)

Arquivos: | Arquivo | Função | |—| | Dockerfile | Imagem Node.js com cache otimizado | | compose.yaml | 2 serviços, healthcheck, volume | | index.js | Rotas / (JSON) e /health |

Notas do apresentador: “Criamos um repositório no GitHub com uma aplicação Node.js + PostgreSQL usando Docker Compose. A arquitetura é simples: dois serviços em uma rede interna isolada. O app se conecta ao banco usando o nome do serviço db como hostname — dentro do Compose, a resolução de DNS é automática. O Dockerfile usa node:20-alpine, uma imagem leve de apenas 130 MB. O compose.yaml declara os dois serviços com healthcheck no banco e volume nomeado para os dados persistirem.”

Slide 9 — Caso Real: Demo

Slide (comandos e output esperado):

```
git clone https://github.com/VDCSS/demo-Docker.git
cd demo-Docker
docker compose up -d --build
curl http://localhost:3000
# {"status":"ok","time":"2026-06-13T..."}
docker compose ps
docker compose logs --tail=5
docker compose down -v
```

Notas do apresentador: “Vamos ver na prática. Primeiro clonamos o repositório. Depois o comando mágico: docker compose up -d --build. O --build força a reconstrução da imagem. Reparem que ele baixa a imagem do PostgreSQL, constrói a imagem da nossa aplicação, cria a rede e os volumes, e sobe tudo. Com docker compose ps vemos os dois serviços rodando. Testamos com curl: a aplicação consulta o banco e retorna um JSON com a hora atual — prova que a stack inteira está funcionando. Damos uma olhada nos logs e depois derrubamos tudo com docker compose down -v.”

Slide 10 — Erros Comuns

Slide (4 erros + soluções):

Erro	Causa	Solução
“Cannot connect to Docker daemon”	Usuário sem grupo docker	sudo usermod -aG docker \$USER + newgrp docker
“port is already allocated”	Porta ocupada	docker ps / lsof, usar -p 3001:3000
ECONNREFUSED no banco	Host errado (localhost em vez do serviço)	Usar nome do serviço (db), não localhost
Imagem não aproveita cache	Ordem errada no Dockerfile	Copiar package.json antes do código

Notas do apresentador: “Erro mais clássico: ‘Cannot connect to the Docker daemon’ — você instalou o Docker, tentou rodar e nada. Provavelmente seu usuário não está no grupo docker. Comando resolve na hora. Segundo: ‘port is already allocated’ — já tem algo na porta. Use docker ps para achar o container conflitante e troque a porta. Terceiro: ECONNREFUSED no banco — dentro do Compose, use o nome do serviço como host, não localhost. Quarto: a imagem não aproveita cache porque o COPY . . vem antes do npm install — copie o package.json primeiro, instale, depois copie o código.”

Slide 11 — Boas Práticas + Referências

Slide (bullet points):

Boas Práticas: - Use imagens Alpine — menores e mais seguras - Tenha um .dockerignore — evita copiar lixo para a imagem - Copie package.json antes do código — aproveita cache de camadas - Nomeie volumes e redes no Compose - Use

healthcheck em serviços críticos (banco, API)

Referências: - docs.docker.com - docs.docker.com/compose - Dockerfile reference
- Docker Hub: hub.docker.com - Play with Docker: labs.play-with-docker.com

QR Code: github.com/VDCSS/demo-Docker

Notas do apresentador: “Boas práticas resumidas: prefira imagens Alpine — são muito menores e com menos superfície de ataque. Sempre tenha um `.dockerignore` para não copiar `node_modules` e `.git` para dentro da imagem. A ordem no Dockerfile importa: copie o `package.json` primeiro, instale as dependências, depois copie o código — isso faz o cache funcionar. No Compose, nomeie volumes e redes explicitamente em vez de usar os nomes automáticos. E use healthcheck em serviços dos quais outros dependem. As referências oficiais estão todas aqui, e o repositório da nossa demo pode ser acessado pelo QR code.”

Slide 12 — Perguntas

Slide: - Obrigado! - github.com/VDCSS/demo-Docker - Perguntas?

Notas do apresentador (preparação para Q&A):

Q1: “Diferença entre docker-compose e docker compose?” R: docker-compose (hífen) é o script Python legado. docker compose (espaço) é o plugin moderno integrado ao Docker CLI desde 2023. Use sempre o segundo.

Q2: “Precisa instalar Node.js e PostgreSQL?” R: Não. Tudo roda dentro dos containers. As imagens oficiais já vêm com tudo configurado. Você só precisa do Docker.

Q3: “Como acessar o banco de fora do container?” R: Por padrão a porta 5432 não é exposta. Se precisar, adicione ports: ["5432:5432"] no service db.

Q4: “Docker é só para Linux?” R: Não. No Windows e Mac, o Docker Desktop usa uma VM Linux leve. Comandos e experiência são idênticos.